RoboCar: Programm anpassen und erweitern

Anleitung für Studierende

Anleitung für das Modul TRAIL – Projektwoche Maschinenbau
zum Projekt *Bau & Battle: Maschinenbau trifft Autofußball*
im Studiengang Bachelor Maschinenbau
an der Fakultät für Anlagen-, Energie- und Maschinensysteme
der Technischen Hochschule Köln

Gilt für:	Firmware 1.1 und die Web-App
App:	https://stberman.github.io/robocar-steuerung/
Quellen:	https://github.com/stberman/robocar-steuerung
Modulverantwortliche:	Prof. Dr. phil. Anja Richert
Verantwortlich für die Projektwoche:	Prof. Dr. Jörg Luderich

Köln, Wintersemester 2026/27

Inhaltsverzeichnis

Abbildungsverzeichnis **III**

1. Überblick: Wie funktioniert die Steuerung? **1**
1.1. Was darf ich ändern? . . . 1
1.2. Die vier Zusatzkanäle AUX 1–4 . . . 3
1.3. Der Weg durch diese Anleitung . . . 3

2. Dateien besorgen **4**
2.1. Was gibt es wo? . . . 5
2.2. Die ZIP-Datei herunterladen und entpacken . . . 6
2.3. Was ist in welchem Ordner? . . . 8

3. Die App benutzen **9**
3.1. App öffnen . . . 9
3.2. Verbinden . . . 10
3.3. Fahren . . . 11
3.4. Einstellungen, Reiter „Allgemein“ . . . 12
3.5. Einstellungen, Reiter „Kalibrieren“ . . . 12

4. Laptop vorbereiten: Arduino IDE **14**
4.1. Arduino IDE installieren . . . 14
4.2. ESP32-Unterstützung installieren (Boardverwalter) . . . 14
4.3. Bluetooth-Bibliothek installieren (Bibliotheksverwalter) . . . 16
4.4. USB-Treiber . . . 16
4.5. Programm öffnen . . . 17

5. Den ESP32 flashen (Programm aufspielen) **18**
5.1. Was heißt „Flashen“? . . . 18
5.2. Vorbereitung . . . 18
5.3. Einstellungen in der Arduino IDE . . . 19
5.4. Hochladen . . . 20
5.5. Kontrolle: Läuft das Programm? . . . 23
5.6. Erste eigene Änderung: der Name Ihres Autos . . . 23
5.7. Notfall: alles zurücksetzen . . . 25

6. Kleiner C++-Crashkurs **26**
6.1. Kommentare . . . 26
6.2. Anweisungen und Semikolon . . . 26
6.3. Variablen und Konstanten . . . 26
6.4. Rechnen und Vergleichen . . . 27
6.5. Entscheidungen: `if` und `else` . . . 27
6.6. Funktionen . . . 28
6.7. Pins ansteuern . . . 28
6.8. Warten, ohne zu blockieren: `millis()` statt `delay()` . . . 28
6.9. Typische Fehlermeldungen des Compilers . . . 30

7. Die App anpassen: eigene Tasten **31**
7.1. Die Stelle in `controls.js` . . . 31
7.2. Weg A: Am Laptop testen . . . 32
7.3. Weg B: Die eigene App aufs Handy bringen (GitHub) . . . 33

8. Projekt: Servo-Kicker **36**
8.1. Was ist ein Servo? . . . 36
8.2. Anschließen . . . 36
8.3. Stufe 1: das fertige Beispiel einschalten . . . 37
8.4. Stufe 2: Winkel einstellen . . . 38
8.5. Stufe 3: Ein Druck = ein Schuss (mit `millis()`) . . . 39
8.6. Stufe 4: Servo stufenlos per Schieber . . . 41
8.7. Mechanik: So kann ein Kicker aussehen . . . 43

9. Eigene Ideen: freie Pins und Grenzen **45**

10. Fehlersuche **47**

A. Spickzettel **49**

B. Verkabelung des Autos **51**

C. Bildnachweis **52**

Abbildungsverzeichnis

1.1. So hängen die Teile zusammen. Das USB-Kabel zum Laptop brauchen Sie nur, wenn Sie ein neues Programm auf den ESP32 spielen. 2

2.1. Oben: der grüne Knopf „Code“ auf der Projektseite (Schritt 2). Unten: das aufgeklappte Menü, ganz unten „Download ZIP“ (Schritt 3) 7
2.2. Inhalt des entpackten Ordners 8

3.1. Startbildschirm der App 10
3.2. Auswahlfenster für das Auto. Hier heißt das Auto `RoboCar-Max Mustermann`; bei Ihnen steht dort der Name Ihres Autos. Bei Android danach auf „Koppeln“ tippen. 10
3.3. Die drei Fahrmodi. Oben in der Leiste stehen Name und Verzögerung (in ms). 11

4.1. Boardverwalter: „esp32“ von Espressif Systems ist installiert 15
4.2. Bibliotheksverwalter mit NimBLE-Arduino 16

5.1. Vom Code zum laufenden Programm 18
5.2. ESP32 am Laptop: Tasten und LEDs 19
5.3. Das Menü „Werkzeuge“: **1** Board, **2** Port (bei Ihnen evtl. eine andere Nummer), **3** Erase All Flash „Disabled“, **4** Upload Speed 21
5.4. Der ESP32 im Geräte-Manager, hier an `COM4`. „COM1“ ist ein interner Anschluss des Laptops und nicht der ESP32. 22
5.5. Erfolgreich hochgeladen: 100 % geschrieben, danach „Hard resetting via RTS pin“ 22
5.6. Der Serielle Monitor: oben rechts das Symbol zum Öffnen, rechts daneben die Einstellung 115200 Baud, unten die Startmeldung. Den Namen Ihres Autos sehen Sie in der Zeile „Bluetooth bereit als …“. . . 24

6.1. Wann Ihre Funktionen (grün) aufgerufen werden 29

7.1. Die geänderte App im Demo-Modus. Oben steht, welche Werte gesendet würden: `Aux 255/0/160/0`. 32
7.2. Der Knopf „Fork“ auf der Projektseite 34

7.3. In Ihrem Fork: **a** Settings, **b** Pages, **c** Deploy from a branch, **d** main, /docs, Save. Oben links steht hier „beispielAccount“, bei Ihnen Ihr eigener Name. 34
7.4. GitHub: Datei controls.js bearbeiten 35
7.5. Nach dem Ändern: oben rechts „Commit changes …“ klicken 35

8.1. Servo am ESP32: Signal an D4 (GPIO 4), Plus an VIN, Minus an GND 37
8.2. Prinzipskizze Servo-Kicker 44

9.1. ESP32 DevKit V1 von oben (USB-Buchse unten). Innen steht die Beschriftung auf dem Board, außen die Bedeutung. Achtung: GPIO 16 und 17 sind auf dem Board mit **RX2** und **TX2** beschriftet. 46

B.1. Verkabelungsplan WS26/27 (aus der Aufbauanleitung) 51

1. Überblick: Wie funktioniert die Steuerung?

Ziel dieses Kapitels: Sie wissen, aus welchen Teilen die Steuerung besteht und welche Dateien Sie ändern dürfen.

● **Für wen ist diese Anleitung?** Für alle, die ihr Auto über das Fahren hinaus anpassen wollen, zum Beispiel mit einem eigenen Namen, eigenen Tasten in der App oder einem Servo-Kicker. **Programmiererfahrung brauchen Sie nicht.** Jeder Schritt ist erklärt. Fahren können Sie auch ohne diese Anleitung (Kapitel 3).

Ihr Auto wird mit Ihrem eigenen Handy gesteuert. Dazu arbeiten zwei Programme zusammen:

- **Die App** läuft im Browser Ihres Handys. Sie zeigt Joystick und Tasten an und schickt 25-mal pro Sekunde per *Bluetooth Low Energy* (BLE) ein kleines Datenpaket an das Auto: wie viel Gas, wie viel Lenkung, welche Tasten gedrückt sind.
- **Die Firmware** ist das Programm auf dem ESP32, dem kleinen Computer auf Ihrem Auto. Sie empfängt die Pakete und steuert über den Motortreiber L298N die vier Motoren.

1.1. Was darf ich ändern?

Beide Programme sind so gebaut, dass Sie an **wenigen, klar markierten Stellen** eigene Ideen einbauen können, ohne die Bluetooth-Verbindung oder die Motorsteuerung kaputtzumachen.

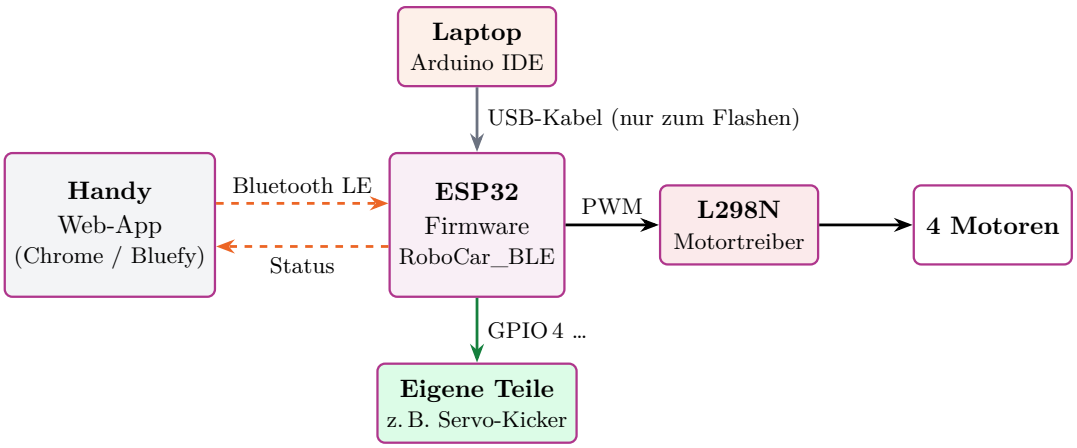


Abbildung 1.1.: So hängen die Teile zusammen. Das USB-Kabel zum Laptop brauchen Sie nur, wenn Sie ein neues Programm auf den ESP32 spielen.

	Datei	Wofür?
Firmware (Ordner **`firmware/RoboCar_BLE/`**, Arduino IDE)		
✓	**`RoboCar_BLE.ino`**	ganz oben: **Name des Autos** (`AUTO_NAME`)
✓	**`aux_user.cpp`**	**Ihre eigenen Funktionen**: Servo, Kicker, Sensoren …
●	**`config.h`**	Pinbelegung, nur lesen (hier stehen die freien Pins)
✗	**`motor.cpp`**, **`ble_link.cpp`**, **`settings.cpp`**	Motorsteuerung, Bluetooth, Speicher, **nicht ändern**
App (Ordner **`docs/`**, beliebiger Texteditor)		
✓	**`controls.js`**	ganz oben: **Namen und Art der Zusatztasten** (`AUX_KANAELE`)
✓	**`style.css`**	Farben und Aussehen (für Fortgeschrittene)
✗	**`ble.js`**	Bluetooth, **nicht ändern** (muss genau zur Firmware passen)

1.2. Die vier Zusatzkanäle AUX 1–4

Für eigene Erweiterungen schickt die App neben Gas und Lenkung noch **vier zusätzliche Werte** mit, die Kanäle **AUX 1 bis AUX 4** (AUX = *auxiliary*, englisch für „zusätzlich“). Jeder Kanal ist eine Zahl von **0 bis 255**. In der App legen Sie fest, wie der Kanal bedient wird:

Art	Verhalten in der App	Wert im Auto
Taster	nur aktiv, solange der Finger drauf ist	0 = losgelassen, 255 = gedrückt
Schalter	jedes Antippen schaltet um	0 = aus, 255 = an
Schieber	stufenlos verschieben	0 … 255

Auf dem ESP32 kommt jeder geänderte Wert in der Funktion `onAux(kanal, wert)` in der Datei **`aux_user.cpp`** an. Dort programmieren Sie, was passieren soll. Ab Werk sind die Kanäle so belegt: AUX 1 = „KICK“ (Taster), AUX 2 = „LICHT“ (Schalter), AUX 3 = „ARM“ (Schieber), AUX 4 = ausgeblendet.

● **Der Weg einer Taste:** Sie tippen in der App auf **KICK** → die App setzt AUX 1 auf 255 → das Paket fliegt per Bluetooth zum Auto → die Firmware merkt „AUX 1 hat sich geändert“ und ruft `onAux(1, 255)` auf → Ihr Code dort bewegt den Servo.

1.3. Der Weg durch diese Anleitung

1. Dateien besorgen: alles in einer ZIP-Datei von GitHub (Kapitel 2).
2. Auto mit der App fahren und einstellen (Kapitel 3). *Das brauchen alle.*
3. Laptop vorbereiten (Kapitel 4) und das Programm einmal selbst aufspielen (Kapitel 5).
4. Ein paar Grundlagen der Programmiersprache C++ (Kapitel 6).
5. Zusatztasten in der App anpassen (Kapitel 7).
6. Projekt Servo-Kicker (Kapitel 8), danach eigene Ideen (Kapitel 9).

Wenn etwas nicht klappt: Kapitel 10 „Fehlersuche“. Der Spickzettel ganz hinten fasst alles auf einer Seite zusammen.

2. Dateien besorgen

Ziel dieses Kapitels: Sie haben alle Dateien für die Projektwoche auf Ihrem Laptop und wissen, was in welchem Ordner liegt.

2.1. Was gibt es wo?

Was?	Woher?	Brauche ich das?
Diese Anleitung und die Aufbauanleitung (PDF)	auf GitHub im Ordner `anleitungen` (https://github.com/stberman/robocar-steuerung/tree/main/anleitungen), außerdem in der ZIP-Datei und bei den Betreuenden	ja
Die App	nichts herunterladen, nur die Adresse https://stberman.github.io/robocar-steuerung/ im Handy-Browser öffnen (Kapitel 3)	ja, zum Fahren
Firmware, App-Dateien, Beispiele	**eine ZIP-Datei** von GitHub (siehe unten)	sobald Sie etwas am Auto oder an der App ändern wollen
Arduino IDE	https://www.arduino.cc/en/software (Kapitel 4)	zum Aufspielen auf den ESP32

● **Was ist GitHub?** GitHub ist eine Internetseite, auf der Programmierer ihre Programme ablegen und mit anderen teilen. Unser Projekt liegt unter https://github.com/stberman/robocar-steuerung. Dort steht immer die **aktuelle Version**. Zum Herunterladen brauchen Sie **kein** GitHub-Konto. Ein Konto brauchen Sie erst, wenn Sie Ihre eigene App ins Internet stellen wollen (Kapitel 7.3).

2.2. Die ZIP-Datei herunterladen und entpacken

1. Am Laptop im Browser die Adresse https://github.com/stberman/robocar-steuerung öffnen.

2. Rechts oben über der Dateiliste auf den grünen Knopf **</> Code** klicken (Abbildung 2.1).

3. Im aufgeklappten Fenster ganz unten auf **Download ZIP** klicken. Der Browser speichert die Datei **`robocar-steuerung-main.zip`** im Ordner „Downloads“. Sie ist etwa 10 MB groß, weil die Anleitungen als PDF mit drin sind.

4. Im Datei-Explorer den Ordner „Downloads“ öffnen. **Windows**: Rechtsklick auf die ZIP-Datei ➙ **Alle extrahieren ...** ➙ als Ziel `C:\RoboCar` eintippen ➙ **Extrahieren**.
   macOS: Doppelklick auf die ZIP-Datei, dann den neuen Ordner z. B. in Ihren Benutzerordner unter `RoboCar` schieben.

5. Kontrolle: Es gibt jetzt den Ordner `C:\RoboCar\robocar-steuerung-main` mit den Unterordnern aus Abbildung 2.2.

▲ **Nicht direkt aus der ZIP-Datei arbeiten!** Windows zeigt den Inhalt einer ZIP-Datei auch ohne Entpacken an. Die Arduino IDE kann dort aber nichts speichern und findet die anderen Dateien nicht. **Immer zuerst entpacken** („Alle extrahieren“).

☞ **Warum `C:\RoboCar`?** Die Arduino IDE kommt mit Umlauten (ä, ö, ü) und manchmal auch mit Leerzeichen im Pfad nicht gut zurecht. Ein Ordner wie `C:\Users\Jürgen Müller\OneDrive\Dokumente` kann deshalb zu seltsamen Fehlern führen. Ein kurzer Pfad direkt auf `C:` ist sicher. Auch OneDrive-Ordner

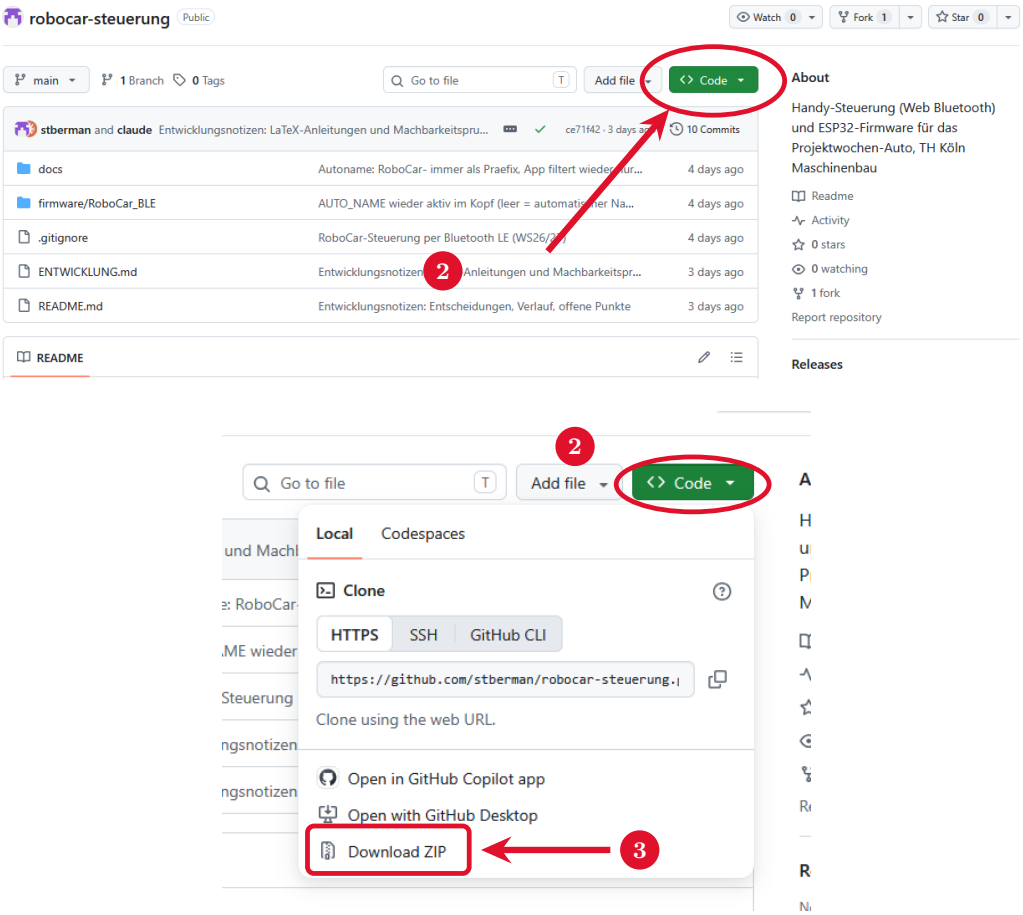


Abbildung 2.1.: Oben: der grüne Knopf „Code“ auf der Projektseite (Schritt 2). Unten: das aufgeklappte Menü, ganz unten „Download ZIP“ (Schritt 3)

meiden: Dort kann die Synchronisation Dateien sperren.

2.3. Was ist in welchem Ordner?

Abbildung 2.2 zeigt den Inhalt des entpackten Ordners und wozu die einzelnen Teile da sind.

`robocar-steuerung-main`	
`anleitungen`	**alle Anleitungen als PDF**: Aufbau, Programm anpassen und flashen (diese hier), Aushang mit QR-Code
`firmware`	
`RoboCar_BLE`	**Firmware** für den ESP32, mit der Arduino IDE öffnen (Kapitel 5)
`RoboCar_BLE.ino`	diese Datei öffnen, die anderen kommen als Reiter mit
`aux_user.cpp`	**Ihre** Erweiterungen (Servo, Licht …)
`config.h`, …	weitere Teile des Programms
`docs`	**App** (Kapitel 7); **`controls.js`** = Ihre Tasten
`beispiele`	fertige Lösungen für das Servo-Kapitel (Kapitel 8)
`README.md`	Kurzbeschreibung des Projekts (für Betreuende)

Abbildung 2.2.: Inhalt des entpackten Ordners

● **Später eine neuere Version holen?** Falls die Betreuenden während der Woche eine neue Version ankündigen, laden Sie die ZIP einfach erneut herunter und entpacken sie in einen **neuen** Ordner, z. B. `C:\RoboCar2`. So überschreiben Sie Ihre eigenen Änderungen nicht. Ihre Änderungen übertragen Sie dann von Hand (meist nur **`aux_user.cpp`** und **`controls.js`**).

3. Die App benutzen

Ziel dieses Kapitels: Sie verbinden Ihr Handy mit dem Auto, fahren und stellen das Auto so ein, dass es sauber geradeaus fährt.

3.1. App öffnen

Android	Adresse https://stberman.github.io/robocar-steuerung/ in **Chrome** öffnen (QR-Code auf dem Aushang). Über Menü (⋮) › Zum Startbildschirm hinzufügen wird daraus ein App-Symbol.
iPhone	Safari kann kein Bluetooth im Browser. Installieren Sie aus dem App Store die kostenlose App **„Bluefy – Web BLE Browser“** und öffnen Sie die Adresse darin.

3.2. Verbinden

1. Auto einschalten. Die **blaue LED** am ESP32 blinkt langsam: Das Auto wartet auf ein Handy.
2. In der App auf **Auto suchen** tippen.
3. Im Auswahlfenster **Ihr** Auto wählen (`RoboCar-` und Ihre Gruppennummer) und „Koppeln“ tippen.
4. Die LED leuchtet jetzt dauerhaft. Das Auto ist verbunden und für alle anderen Handys **unsichtbar**.

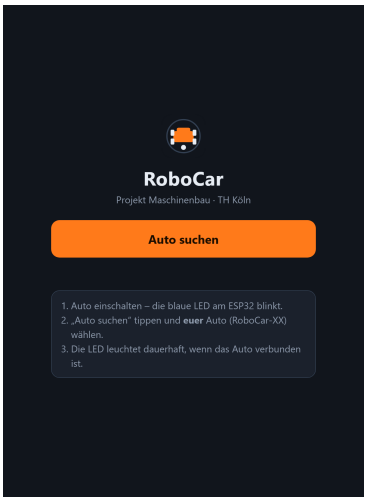
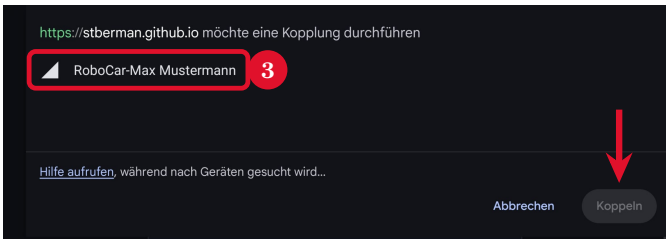
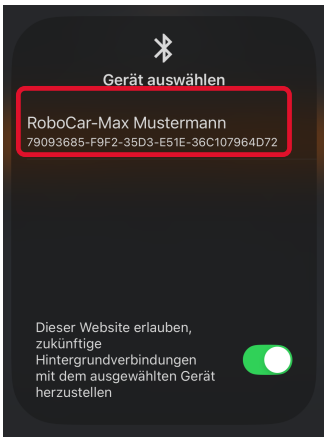


Abbildung 3.1.: Startbildschirm der App



Android (Chrome)



iPhone (Bluefy)

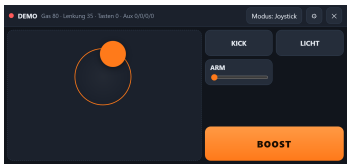
Abbildung 3.2.: Auswahlfenster für das Auto. Hier heißt das Auto `RoboCar-Max Mustermann`; bei Ihnen steht dort der Name Ihres Autos. Bei Android danach auf „Koppeln“ tippen.

☞ **Tipp:** Beim allerersten Start heißt das Auto noch `RoboCar-` mit vier Zeichen dahinter (z. B. `RoboCar-A3F1`). Tragen Sie dann unter ⚙ Allgemein Gruppe Ihre

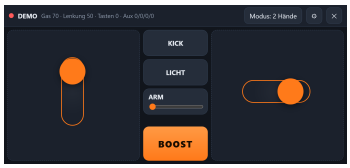
Gruppennummer ein und tippen Sie auf **Speichern**. Das Auto startet neu, heißt danach z. B. `RoboCar-07`, und die App verbindet sich von selbst wieder. Wer ist welches Auto? ⚙ › LED blinken lassen lässt die LED Ihres Autos schnell blinken.

3.3. Fahren

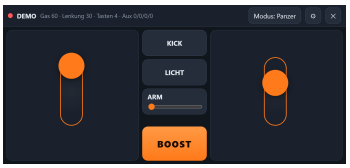
Halten Sie das Handy **quer**. Wo Ihr Daumen aufsetzt, ist die Mitte des Joysticks. Loslassen heißt: Das Auto hält an. Mit der Taste **Modus** oben wechseln Sie zwischen drei Fahrweisen:



Joystick: ein Daumen; hoch/runter = Gas, links/rechts = Lenken



2 Hände: links nur Gas, rechts nur Lenken



Panzer: linker Daumen = linke Seite, rechter Daumen = rechte Seite

Abbildung 3.3.: Die drei Fahrmodi. Oben in der Leiste stehen Name und Verzögerung (in ms).

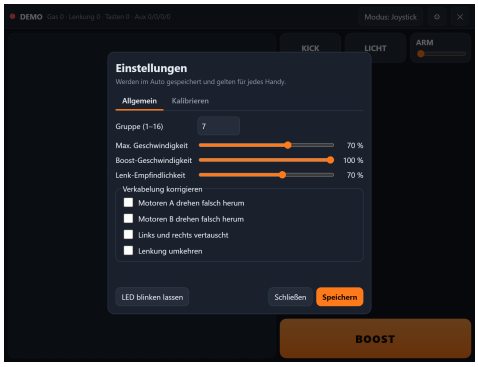
- **BOOST** gedrückt halten: Das Auto fährt mit der Boost-Geschwindigkeit statt der normalen Höchstgeschwindigkeit.
- Die Tasten **KICK**, **LICHT**, **ARM** sind die Zusatzkanäle AUX 1–3. Sie tun erst etwas, wenn Sie am Auto etwas angeschlossen und programmiert haben (Kapitel 8).
- Ein Bluetooth-Gamecontroller am Handy geht auch: linker Stick = Fahren, rechter Trigger = Boost, A/B/X/Y = AUX 1–4.

▲ **Sicherheit:** Kommt 0,3 Sekunden lang kein Paket vom Handy an (Verbindung weg, App gewechselt, Bildschirm aus), bleiben die Motoren sofort stehen. Trotzdem gilt: Beim Testen auf dem Tisch das Auto **aufbocken**, damit es nicht herunterfährt.

3.4. Einstellungen, Reiter „Allgemein“

Über das Zahnrad [⚙] öffnen Sie die Einstellungen. Sie werden **im Auto gespeichert**, gelten also auch, wenn ein anderes Handy fährt, und bleiben beim Ausschalten und beim Aufspielen eines neuen Programms erhalten. Im Reiter „Allgemein“ werden Änderungen erst mit **Speichern** übertragen.

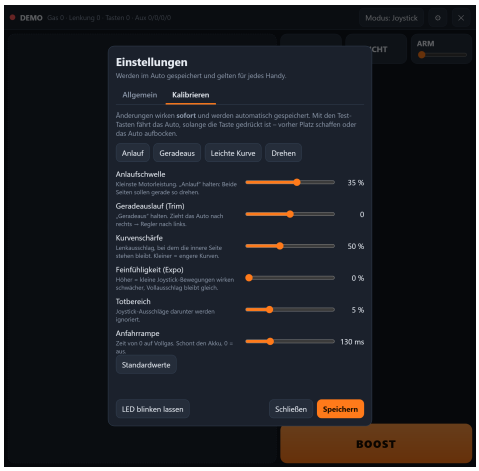
Problem	Lösung
Ein Motorpaar dreht falsch herum	„Motoren A/B drehen falsch herum“
Joystick links, Auto fährt rechts	„Links und rechts vertauscht“
Lenkung genau verkehrt	„Lenkung umkehren“
zu schnell/zu langsam	Max.- und Boost-Geschwindigkeit
Lenkung zu nervös	Lenk-Empfindlichkeit kleiner



> ☞ **Tipp:** Falsch herum angeschlossene Motoren müssen Sie **nicht** umstecken. Ein Häkchen in der App reicht.

3.5. Einstellungen, Reiter „Kalibrieren“

Kein Motor ist wie der andere. Im Reiter „Kalibrieren“ passen Sie die Fahrlogik an **Ihr** Auto an. Hier wirkt jede Änderung **sofort** und wird automatisch gespeichert. Mit den vier Test-Tasten fährt das Auto, **solange Sie die Taste gedrückt halten**.



So gehen Sie vor (Auto aufbocken oder viel Platz):

1. **Anlauf** gedrückt halten. **Anlaufschwelle** so einstellen, dass sich beide Seiten gerade eben drehen.
2. Auto auf den Boden, **Geradeaus** halten. Zieht es nach rechts ➙ **Trim** nach links schieben (und umgekehrt).
3. **Leichte Kurve** und **Drehen** ausprobieren. Ist die Kurve zu weit ➙ **Kurvenschärfe** kleiner.
4. Nach Gefühl: **Expo**, **Totbereich** und **Anfahrrampe**.

Regler	Standard	Wirkung
Anlaufschwelle	35 %	kleinste Motorleistung; darunter brummen die Motoren nur, ohne zu drehen
Geradeauslauf (Trim)	0	bremst eine Seite etwas ab, damit das Auto nicht zieht
Kurvenschärfe	50 %	bei diesem Lenkausschlag bleibt die kurveninnere Seite stehen; kleiner = engere Kurven
Feinfühligkeit (Expo)	0 %	höher = kleine Joystick-Bewegungen wirken schwächer, Vollausschlag bleibt gleich
Totbereich	5 %	so kleine Ausschläge werden ignoriert (gegen „Kriechen“)
Anfahrrampe	130 ms	Zeit von 0 auf Vollgas; schont Akku und Getriebe, 0 = aus

Mit **Standardwerte** stellen Sie alles wieder auf die Werte ab Werk zurück.

4. Laptop vorbereiten: Arduino IDE

Ziel dieses Kapitels: Auf Ihrem Laptop ist alles installiert, um Programme auf den ESP32 zu spielen. **Am besten schon vor der Projektwoche erledigen**, denn der Download ist mehrere hundert MB groß.

Die **Arduino IDE** ist ein kostenloses Programm, mit dem man Programme für kleine Mikrocontroller schreibt und auf sie überträgt (IDE = *Integrated Development Environment*, Entwicklungsumgebung). Sie läuft unter Windows, macOS und Linux.

4.1. Arduino IDE installieren

1. https://www.arduino.cc/en/software öffnen und die **Arduino IDE 2.x** für Ihr Betriebssystem herunterladen („Just download“ reicht, eine Spende ist freiwillig).
2. Installieren und starten. Unter Windows bei Nachfragen nach Treibern immer „Installieren“ wählen.
3. Sprache auf Deutsch stellen (falls gewünscht): File » Preferences » Language: Deutsch.

4.2. ESP32-Unterstützung installieren (Boardverwalter)

Ab Werk kennt die Arduino IDE den ESP32 noch nicht. Das Paket dafür holen Sie im **Boardverwalter**.

1. Links in der Seitenleiste das zweite Symbol (**Boardverwalter**, sieht aus wie eine Platine) anklicken.
2. In das Suchfeld `esp32` eingeben.
3. Beim Eintrag **„esp32 by Espressif Systems“** eine Version **3.x** wählen (getestet: 3.3.12) und **Installieren** klicken. Achtung: *nicht* „Arduino ESP32 Boards“, das ist ein anderes Paket.

4 Warten, bis „INSTALLED“ erscheint. Das kann einige Minuten dauern.

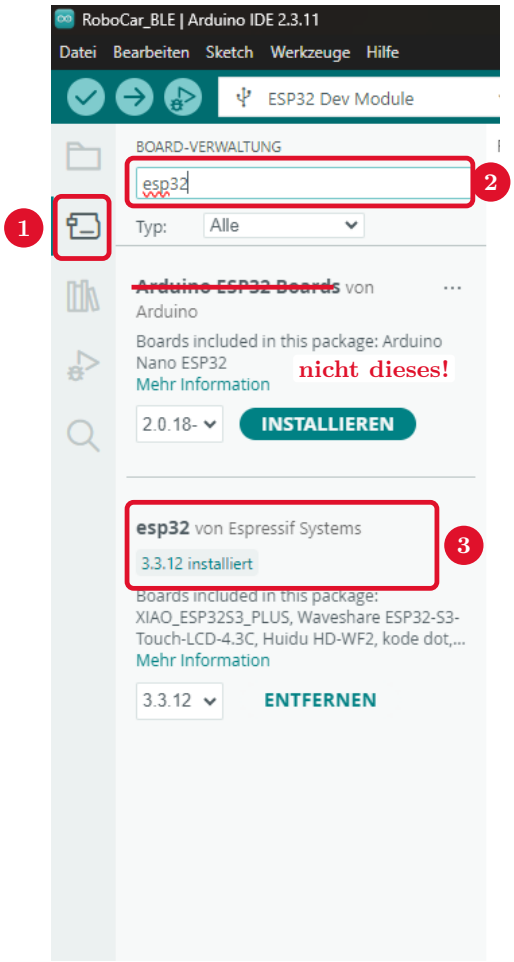


Abbildung 4.1.: Boardverwalter: „esp32“ von Espressif Systems ist installiert

☞ **Paket nicht gefunden?** Unter Datei › Einstellungen › Zusätzliche Boardverwalter-URLs diese Adresse eintragen und neu suchen: https://espressif.github.io/arduino-esp32/package_esp32_index.json

4.3. Bluetooth-Bibliothek installieren (Bibliotheksverwalter)

Die Firmware nutzt für Bluetooth die Bibliothek **NimBLE-Arduino**. Eine Bibliothek ist fertiger Code, den andere geschrieben haben.

1. Links das dritte Symbol (**Bibliotheksverwalter**, sieht aus wie Bücher) anklicken.
2. `NimBLE` suchen.
3. Bei **„NimBLE-Arduino“ von h2zero** eine Version **2.x** wählen (getestet: 2.5.2) und **Installieren**.

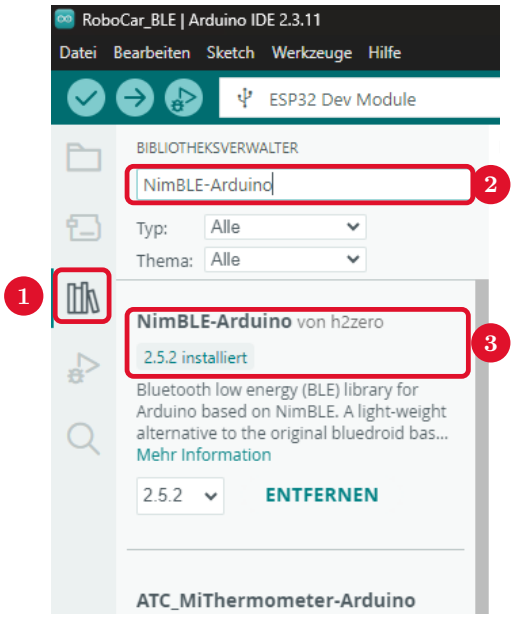


Abbildung 4.2.: Bibliotheksverwalter mit NimBLE-Arduino

4.4. USB-Treiber

Auf dem ESP32-Board sitzt neben der USB-Buchse ein kleiner Chip, der USB in Signale für den ESP32 übersetzt. Je nach Hersteller ist das ein **CP2102** (Aufdruck „SiLabs“ oder „CP2102“) oder ein **CH340**. Windows 10/11 installiert den Treiber meist von selbst. Falls nicht (siehe Kapitel 10):

- CP2102: https://www.silabs.com/developer-tools/usb-to-uart-bridge-vcp-drivers
- CH340: https://www.wch-ic.com/downloads/CH341SER_EXE.html (Windows) bzw. `CH34XSER_MAC` (macOS)

4.5. Programm öffnen

Das Programm haben Sie schon in Kapitel 2 als ZIP heruntergeladen und entpackt.

1. In der Arduino IDE Datei » Öffnen wählen.
2. Zur Datei `C:\RoboCar\robocar-steuerung-main\firmware\RoboCar_BLE\RoboCar_BLE.ino` gehen und öffnen.

Oben sehen Sie jetzt mehrere **Reiter**, einen pro Datei: **`RoboCar_BLE.ino`**, **`aux_user.cpp`**, **`config.h`** usw.

● **Gut zu wissen:** Ein Arduino-Programm ist ein **Ordner**, und der Ordner muss genauso heißen wie die `.ino`-Datei darin (hier `RoboCar_BLE`). Benennen Sie den Ordner deshalb nicht um.

5. Den ESP32 flashen (Programm aufspielen)

Ziel dieses Kapitels: Sie spielen das Programm selbst auf den ESP32, prüfen, dass alles läuft, und geben Ihrem Auto einen eigenen Namen.

5.1. Was heißt „Flashen“?

Der ESP32 speichert sein Programm in einem **Flash-Speicher**, ähnlich wie ein USB-Stick. Der Speicher behält seinen Inhalt auch ohne Strom. „Flashen“ heißt, ein neues Programm in diesen Speicher zu schreiben. Das passiert in zwei Schritten:

1. **Kompilieren**: Ein Programm namens *Compiler* übersetzt Ihren lesbaren Code in Maschinencode, den der Prozessor versteht. Dabei findet er auch Tippfehler.
2. **Hochladen**: Der Maschinencode wird über das USB-Kabel in den Flash-Speicher geschrieben.

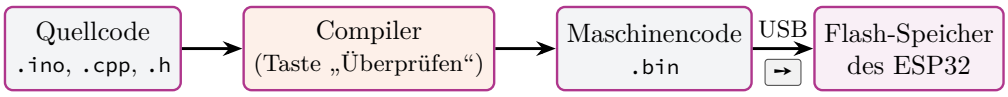


Abbildung 5.1.: Vom Code zum laufenden Programm

● **Ihre Einstellungen bleiben erhalten:** Gruppe, Geschwindigkeit, Trim und Kalibrierung liegen in einem eigenen Speicherbereich. Beim normalen Hochladen werden sie **nicht** gelöscht. Sie können also ohne Sorge neue Versionen ausprobieren.

5.2. Vorbereitung

1. Den Akku-Schalter des Autos **ausschalten** und das Auto aufbocken (Räder in der Luft).

2. Das USB-Kabel der Powerbank vom ESP32 abziehen.

3. ESP32 mit einem USB-Kabel an den Laptop anschließen. Es muss ein **Datenkabel** sein: Manche billigen Kabel können nur laden, dann erscheint kein Port. Im Zweifel ein anderes Kabel probieren.

4. Die rote LED am ESP32 (Strom) leuchtet (Abbildung 5.2).

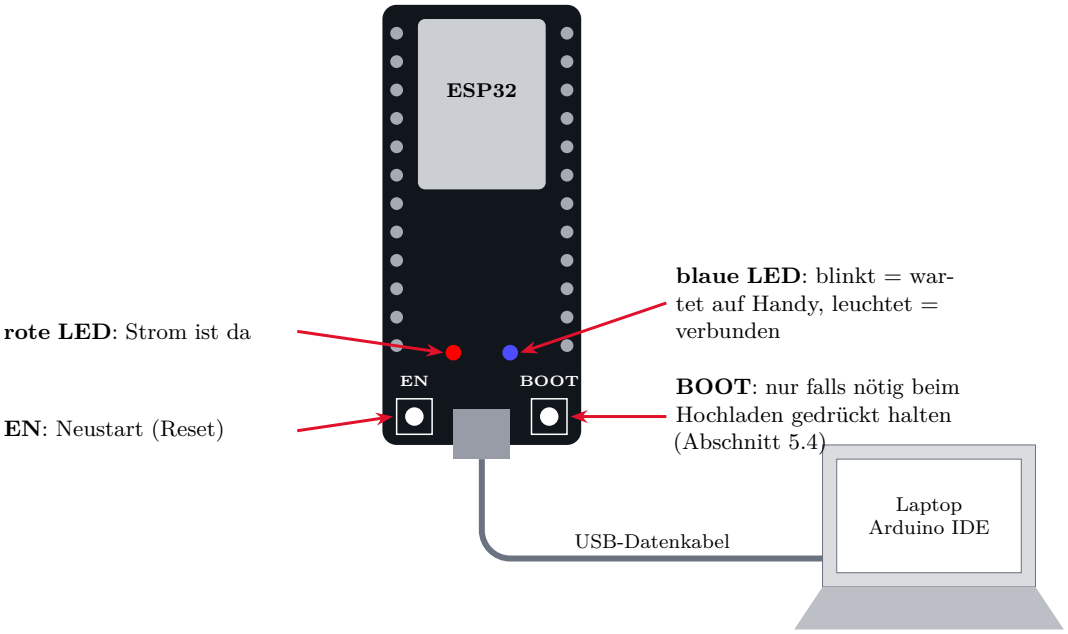


Abbildung 5.2.: ESP32 am Laptop (vereinfachte Zeichnung, USB-Buchse unten). Die Tasten **EN** und **BOOT** sitzen links und rechts neben der USB-Buchse.

5.3. Einstellungen in der Arduino IDE

Diese Einstellungen machen Sie einmal. Die IDE merkt sie sich.

Menü	Einstellung
Werkzeuge › Board › esp32	**ESP32 Dev Module**
Werkzeuge › Port	der Port Ihres ESP32: Windows `COM3`, `COM5` …, macOS `/dev/cu.usbserial-…` oder `/dev/cu.SLAB_USBtoUART`, Linux `/dev/ttyUSB0`
Werkzeuge › Upload Speed	`921600` (bei Problemen `115200`)
Werkzeuge › Erase All Flash Before Sketch Upload	**Disabled** (sonst sind Gruppe und Kalibrierung weg)

☞ **Welcher Port ist der richtige?** Ziehen Sie das USB-Kabel ab und öffnen Sie Werkzeuge › Port. Dann stecken Sie es wieder an und schauen erneut. Der Port, der neu dazugekommen ist, gehört zum ESP32. Unter Windows sehen Sie ihn auch im **Geräte-Manager** (Rechtsklick auf Start) unter „Anschlüsse (COM & LPT)“.

5.4. Hochladen

Oben links in der Arduino IDE gibt es zwei runde Tasten:



Überprüfen: nur kompilieren und Fehler suchen (nichts wird aufgespielt)



Hochladen: kompilieren *und* auf den ESP32 schreiben

1. Auf **Hochladen** klicken.
2. Unten im Ausgabefenster läuft Text durch. Das erste Kompilieren dauert 1–3 Minuten, danach geht es schneller.
3. Dann erscheint `Connecting....`, danach Prozentzahlen (`Writing at 0x… (35 %)`).
4. Fertig ist es bei der Meldung `Hard resetting via RTS pin...`. Der ESP32 startet jetzt mit dem neuen Programm.

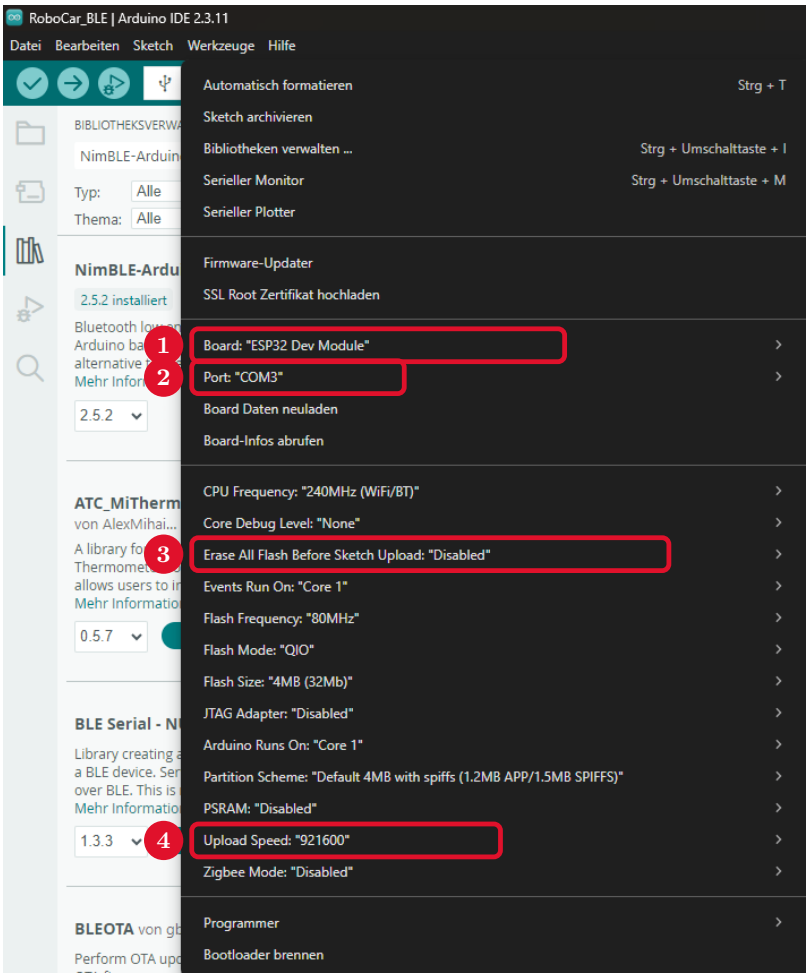


Abbildung 5.3.: Das Menü „Werkzeuge“: **1** Board, **2** Port (bei Ihnen evtl. eine andere Nummer), **3** Erase All Flash „Disabled“, **4** Upload Speed

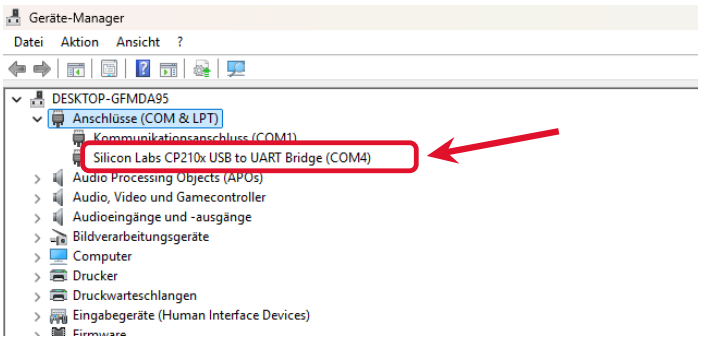


Abbildung 5.4.: Der ESP32 im Geräte-Manager, hier an `COM4`. „COM1“ ist ein interner Anschluss des Laptops und nicht der ESP32.

Listing 5.1: So sieht ein erfolgreiches Hochladen aus (gekürzt)

```
Sketch uses 605795 bytes (46%) of program storage space.
Connecting....
Chip type:          ESP32-D0WD-V3 (revision v3.1)
Uploading stub flasher...
Writing at 0x00010000 [=====>                          ]  18.6% ...
Wrote 605952 bytes (...) at 0x00010000 in 9.1 seconds
Hash of data verified.
Hard resetting via RTS pin...
```

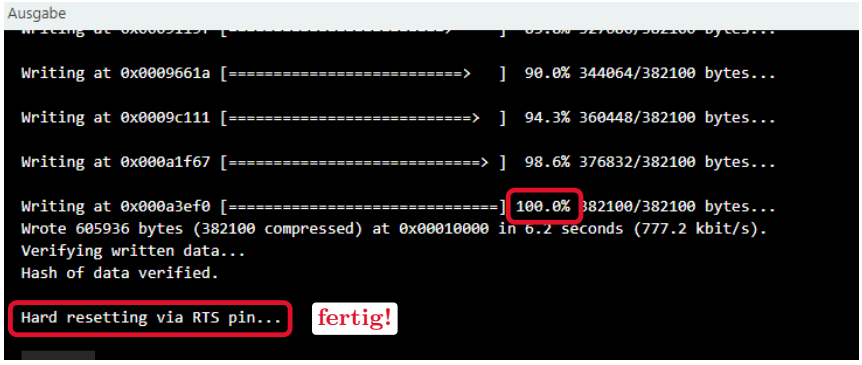


Abbildung 5.5.: Erfolgreich hochgeladen: 100 % geschrieben, danach „Hard resetting via RTS pin“

▲ **Failed to connect to ESP32 oder Wrong boot mode?** Manche Boards gehen nicht von selbst in den Flash-Modus. Dann so:

1. Erneut auf **Hochladen** klicken.

2. Sobald unten `Connecting...` erscheint, die Taste **BOOT** auf dem ESP32 drücken und gedrückt halten.
3. Loslassen, sobald die Prozentzahlen laufen.

Klappt es dauerhaft nicht: anderes USB-Kabel, anderen USB-Anschluss, Upload Speed `115200`.

5.5. Kontrolle: Läuft das Programm?

1. Werkzeuge » Serieller Monitor öffnen (oder Lupe oben rechts).
2. Unten rechts im Monitor **115200 baud** einstellen.
3. Die Taste **EN** auf dem ESP32 kurz drücken (Neustart). Im Monitor erscheint `RoboCar_BLE v1.1 gestartet`.
4. Die blaue LED blinkt langsam. Mit der App verbinden und ein paar Tasten drücken. Im Monitor erscheinen Zeilen wie `AUX 1 = 255`.

● **Was ist der Serielle Monitor?** Über das USB-Kabel kann der ESP32 auch Text an den Laptop schicken. Im Programm steht dafür z. B. `Serial.println("Hallo");`. Das ist Ihr wichtigstes Werkzeug zur Fehlersuche: Lassen Sie sich Werte ausgeben, dann sehen Sie, was im Programm passiert. Der Monitor ist nur nutzbar, solange das USB-Kabel steckt.

5.6. Erste eigene Änderung: der Name Ihres Autos

✎ **Aufgabe 1: Eigener Autoname**

1. Den Reiter **`RoboCar_BLE.ino`** öffnen. Ganz oben steht:

```
#define AUTO_NAME ""
```

2. Zwischen die Anführungszeichen Ihren Namen schreiben, z. B.:

```
#define AUTO_NAME "Blitz"
```

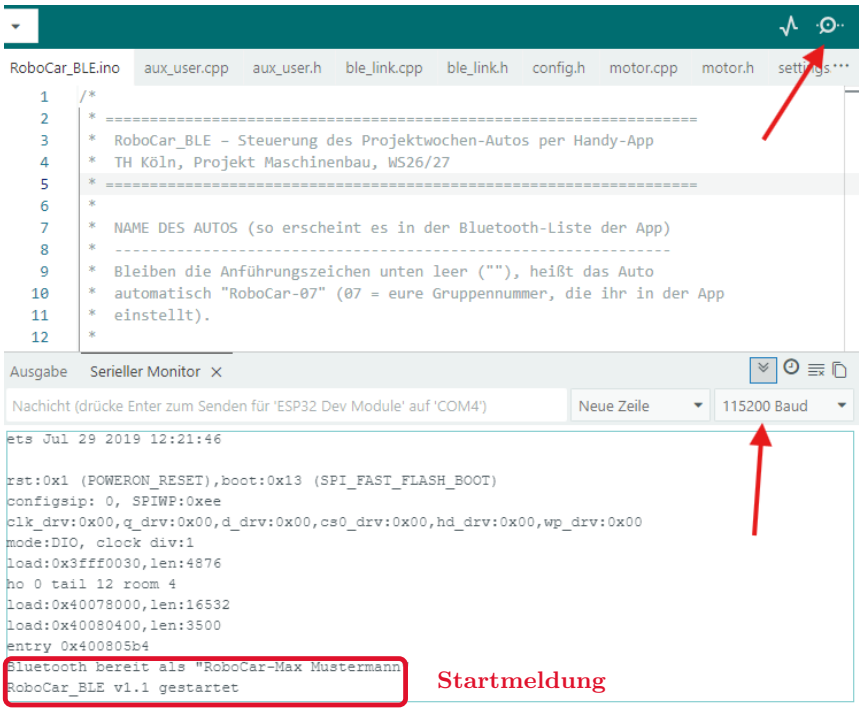


Abbildung 5.6.: Der Serielle Monitor: oben rechts das Symbol zum Öffnen, rechts daneben die Einstellung 115200 Baud, unten die Startmeldung. Den Namen Ihres Autos sehen Sie in der Zeile „Bluetooth bereit als …“.

3 Hochladen. In der App heißt Ihr Auto jetzt `RoboCar-Blitz`.

Regeln: höchstens 18 Zeichen, keine Umlaute (ä, ö, ü, ß) und keine Sonderzeichen. Jedes Auto im Raum braucht einen anderen Namen. Das `RoboCar-` davor setzt das Programm selbst, denn die App zeigt nur Autos mit diesem Anfang. Leere Anführungszeichen bedeuten wieder: Name aus der Gruppennummer.

☞ **Tipp:** Das Handy merkt sich manchmal den alten Namen. Wenn der neue Name nicht erscheint: Bluetooth am Handy aus- und wieder einschalten oder die Seite neu laden.

5.7. Notfall: alles zurücksetzen

Wenn gar nichts mehr geht, spielen Sie das **Originalprogramm** wieder auf: ZIP-Datei von GitHub neu herunterladen (Kapitel 2), in einen neuen Ordner entpacken, **`RoboCar_BLE.ino`** öffnen, hochladen.

Sollen auch alle gespeicherten Einstellungen weg („Werkszustand“), stellen Sie im Menü Werkzeuge den Punkt „Erase All Flash Before Sketch Upload“ einmalig auf **Enabled**, laden hoch und stellen ihn danach **wieder auf Disabled**. Danach müssen Sie Gruppe und Kalibrierung in der App neu einstellen.

6. Kleiner C++-Crashkurs

Ziel dieses Kapitels: Sie können die Datei **`aux_user.cpp`** lesen und verstehen, was jede Zeile ungefähr tut.

Die Firmware ist in **C++** geschrieben, der Sprache der Arduino IDE. Für Ihre Erweiterungen brauchen Sie nur wenige Bausteine.

6.1. Kommentare

Alles hinter `//` bis zum Zeilenende und alles zwischen `/*` und `*/` ist ein **Kommentar**. Der Compiler ignoriert es. Kommentare sind Notizen für Menschen. Schreiben Sie viele davon!

```
// Das ist ein Kommentar
/* Das ist ein Kommentar
   über mehrere Zeilen */
```

6.2. Anweisungen und Semikolon

Jede Anweisung endet mit einem **Semikolon `;`**. Das vergessene Semikolon ist der häufigste Fehler überhaupt. Groß- und Kleinschreibung zählt: `digitalWrite` ist nicht `digitalwrite`.

6.3. Variablen und Konstanten

Eine Variable ist ein benannter Speicherplatz für einen Wert. Vor dem Namen steht der **Typ**:

```
int winkel = 90;                    // ganze Zahl (auch negativ)
uint8_t wert = 255;                 // ganze Zahl 0..255 (1 Byte), so kommen
    ↪ die AUX-Werte an
bool schussLaeuft = false;          // Wahrheitswert: true (wahr) oder false
    ↪ (falsch)
unsigned long startZeit = 0;        // große Zahl >= 0, für Zeiten in
    ↪ Millisekunden
const int SERVO_PIN = 4;            // const = Konstante, kann sich nie ä
    ↪ ndern
```

`#define NAME wert` ist eine andere Art, einer festen Zahl einen Namen zu geben. Der Compiler ersetzt dann überall `NAME` durch `wert`.

6.4. Rechnen und Vergleichen

`a + b, a - b, a * b, a / b`	Grundrechenarten	`a == b`	gleich (zwei =!)
`x = 5;`	zuweisen	`a != b`	ungleich
`a % b`	Rest der Division	`a > b, a <= b`	größer, kleiner-gleich
`!x`	nicht	`a && b, a || b`	und, oder

▲ **Achtung:** `=` weist einen Wert zu, `==` vergleicht. `if (kanal = 1)` ist ein Fehler, der trotzdem kompiliert und dann immer zutrifft. Richtig ist `if (kanal == 1)`.

6.5. Entscheidungen: `if` und `else`

```
if (wert > 127) {
  // passiert, wenn die Bedingung wahr ist (Taste gedrückt)
} else {
  // passiert sonst
}
```

Die geschweiften Klammern `{ }` fassen mehrere Anweisungen zu einem **Block** zusammen. Zu jeder öffnenden Klammer gehört eine schließende. Die Arduino IDE hebt das Gegenstück hervor, wenn Sie den Cursor neben eine Klammer setzen.

6.6. Funktionen

Eine Funktion ist ein benanntes Stück Programm. In den runden Klammern stehen die **Parameter**, also die Werte, die die Funktion bekommt:

```
void onAux(uint8_t kanal, uint8_t wert) {   // void = gibt nichts zurück
  // kanal und wert kann man hier drin benutzen
}
```

Sie **schreiben** die Funktionen in **`aux_user.cpp`**, aber das Hauptprogramm **ruft sie auf**:

`userSetup()`	einmal beim Einschalten: Pins einrichten, Startstellung
`onAux(kanal, wert)`	jedes Mal, wenn sich ein AUX-Wert in der App ändert
`userLoop()`	ständig, alle 10 ms (auch wenn keine Verbindung besteht!)
`onFailsafe()`	wenn die Verbindung abbricht: alles in einen sicheren Zustand bringen

6.7. Pins ansteuern

Ein **GPIO** (*General Purpose Input/Output*) ist ein Anschluss des ESP32, den das Programm ein- und ausschalten oder abfragen kann.

```
pinMode(4, OUTPUT);          // Pin 4 als Ausgang einrichten (in
    ↪ userSetup)
digitalWrite(4, HIGH);       // Pin 4 einschalten (3,3 V)
digitalWrite(4, LOW);        // Pin 4 ausschalten (0 V)

pinMode(23, INPUT_PULLUP);   // Pin 23 als Eingang, z. B. für einen
    ↪ Taster gegen GND
bool gedrueckt = (digitalRead(23) == LOW);
```

6.8. Warten, ohne zu blockieren: `millis()` statt `delay()`

In vielen Arduino-Beispielen im Internet steht `delay(1000);` (eine Sekunde warten). **Benutzen Sie das hier nicht!** Während `delay()` wartet, tut der ESP32 *nichts anderes*: Er bekommt keine Steuerdaten und reagiert nicht auf das Handy.

Stattdessen merkt man sich die Uhrzeit und schaut in `userLoop()` regelmäßig nach, ob genug Zeit vergangen ist. `millis()` liefert die Millisekunden seit dem Einschalten:

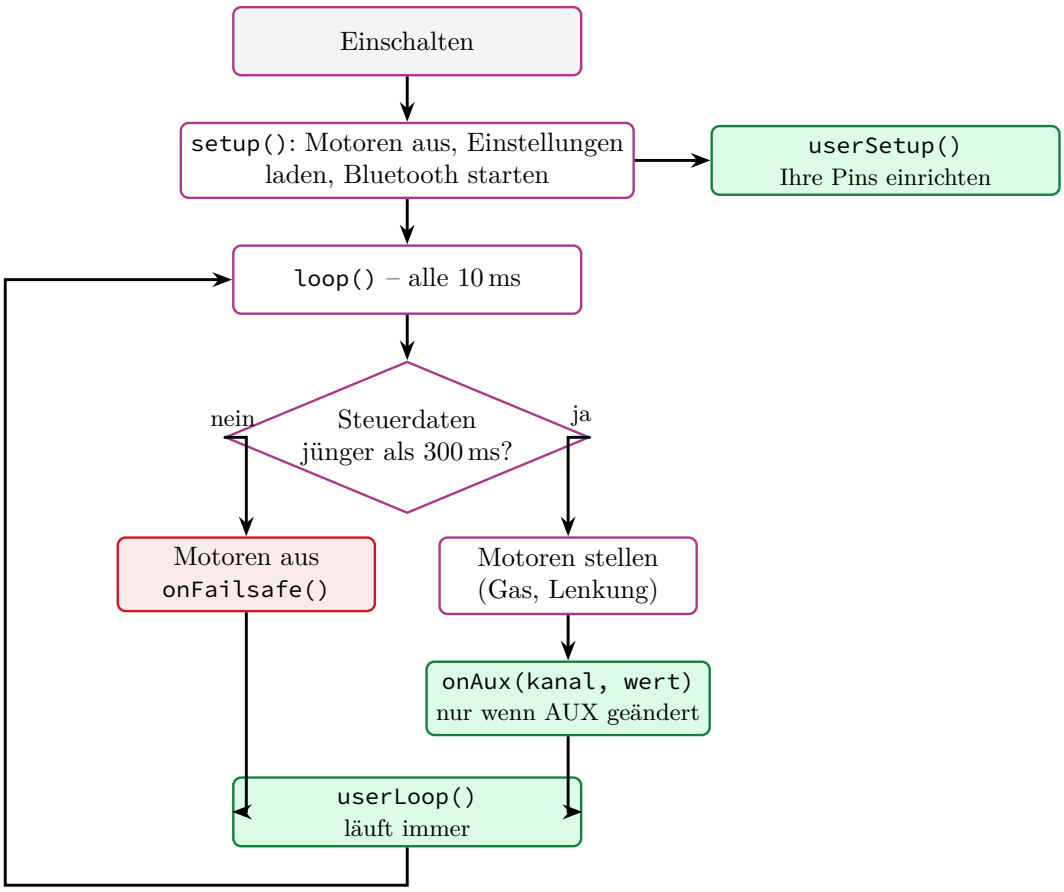


Abbildung 6.1.: Wann Ihre Funktionen (grün) aufgerufen werden

```
unsigned long start = 0;     // oben in der Datei, außerhalb jeder
    ↪ Funktion

// wenn es losgehen soll:
start = millis();

// in userLoop():
if (millis() - start >= 500) {
  // 500 ms sind vorbei -> nächster Schritt
}
```

Das ist wie ein Blick auf die Uhr, statt neben dem Herd zu stehen und zu warten.

6.9. Typische Fehlermeldungen des Compilers

Meldung	Bedeutung
`expected ';' before …`	Semikolon in der Zeile *davor* vergessen
`'xyz' was not declared in this scope`	Name falsch geschrieben (Groß-/Kleinschreibung!) oder Variable nicht angelegt
`expected '}' at end of input`	eine geschweifte Klammer zu wenig
`redefinition of 'void servoGrad(int)'`	Funktion doppelt vorhanden, z. B. Beispiel eingeschaltet *und* selbst kopiert

Die IDE markiert die Zeile rot. Der Fehler steht oft **in der Zeile davor**. Lesen Sie immer die **erste** Fehlermeldung, die folgenden sind oft nur Folgefehler.

7. Die App anpassen: eigene Tasten

Ziel dieses Kapitels: Sie geben den Zusatztasten in der App eigene Namen und Funktionen und testen das zuerst am Laptop, dann auf dem Handy.

Die App besteht aus normalen Textdateien (HTML, CSS, JavaScript) im Ordner **`docs/`**. Sie brauchen keinen besonderen Editor. Notepad geht, angenehmer ist z. B. **Visual Studio Code** (kostenlos).

7.1. Die Stelle in `controls.js`

Ganz oben in **`docs/controls.js`** steht die Liste der vier AUX-Kanäle:

```
const AUX_KANAELE = [
  { name: 'KICK',  art: 'taster'   },   // AUX 1
  { name: 'LICHT', art: 'schalter' },   // AUX 2
  { name: 'ARM',   art: 'schieber' },   // AUX 3
  { name: 'AUX 4', art: null       },   // AUX 4
];
```

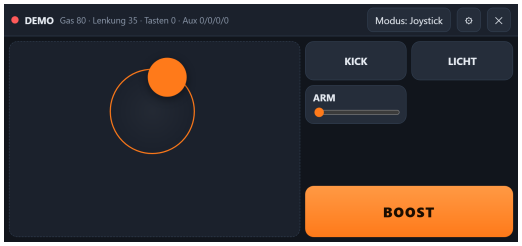
- `name`: die Beschriftung der Taste (kurz halten, ca. 6 Zeichen passen gut).
- `art`: `'taster'`, `'schalter'`, `'schieber'` oder `null` (= nicht anzeigen).
- Die **Reihenfolge** bestimmt den Kanal: Der erste Eintrag ist AUX 1, der zweite AUX 2 usw.

▲ Achtung: In JavaScript müssen Texte in Hochkommas `'...'` stehen, und zwischen den Einträgen steht ein Komma. Fehlt eines davon, bleibt die App nach dem Laden **leer**. Dann Datei prüfen und neu laden.

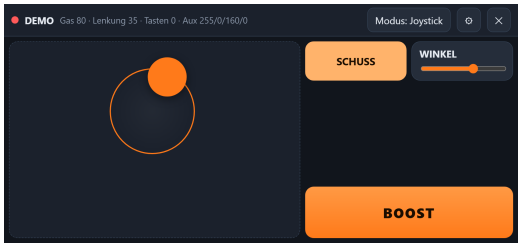
✎ Aufgabe 2: Tasten für den Servo-Kicker

Ändern Sie die Liste so, dass es einen Taster „SCHUSS“ auf AUX 1 und einen Schieber „WINKEL“ auf AUX 3 gibt und AUX 2 nicht mehr angezeigt wird:

```
const AUX_KANAELE = [
  { name: 'SCHUSS', art: 'taster'   },   // AUX 1
  { name: 'LICHT',  art: null       },   // AUX 2
  { name: 'WINKEL', art: 'schieber' },   // AUX 3
  { name: 'AUX 4',  art: null       },   // AUX 4
];
```



vorher



nachher (SCHUSS gedrückt, WINKEL auf 160)

Abbildung 7.1.: Die geänderte App im Demo-Modus. Oben steht, welche Werte gesendet würden: `Aux 255/0/160/0`.

7.2. Weg A: Am Laptop testen

Der schnellste Weg, um Änderungen auszuprobieren.

Nur ansehen (Demo-Modus, ohne Auto):

1. Die Datei **`docs/index.html`** in **Chrome** oder **Edge** öffnen (Doppelklick oder in das Fenster ziehen).
2. In der Adresszeile ganz hinten `?demo` anhängen und Enter drücken, also z. B. `file:///C:/RoboCar/…/docs/index.html?demo`
3. Die Fahransicht erscheint. Oben stehen die Werte, die gesendet würden. Klicken Sie Ihre Tasten an und prüfen Sie die AUX-Werte.
4. Nach jeder Änderung an der Datei: speichern und im Browser F5 drücken.

Mit dem Auto verbinden (Laptop mit Bluetooth): Bluetooth im Browser funktioniert aus Sicherheitsgründen nur über `https://` oder `http://localhost`, nicht über `file://`. Dafür starten Sie einen kleinen Webserver auf dem Laptop:

1. Python installieren, falls noch nicht vorhanden (https://www.python.org).
2. Im Datei-Explorer den Ordner **`docs`** öffnen, in die Adresszeile `cmd` tippen und Enter drücken.
3. Im schwarzen Fenster eingeben:

```
python -m http.server 8000
```

4. In Chrome `http://localhost:8000` öffnen. Jetzt funktioniert **Auto suchen** wie am Handy.
5. Zum Beenden das schwarze Fenster schließen.

7.3. Weg B: Die eigene App aufs Handy bringen (GitHub)

Damit Ihr Handy die geänderte App laden kann, muss sie im Internet unter `https://` erreichbar sein. Das geht kostenlos mit **GitHub Pages**. Sie legen eine eigene Kopie des Projekts an (einen *Fork*) und bearbeiten die Dateien direkt im Browser. Sie müssen nichts installieren.

1. **Konto anlegen**: auf https://github.com registrieren (eine Person pro Gruppe genügt).
2. **Fork erstellen**: https://github.com/stberman/robocar-steuerung öffnen und oben rechts auf **Fork → Create fork** klicken. Sie haben jetzt eine eigene Kopie unter `github.com/`*`IhrName`*`/robocar-steuerung`.
3. **Pages einschalten**: In *Ihrem* Fork Settings › Pages. Unter „Build and deployment“: Source **Deploy from a branch**, Branch **main**, Ordner **/docs**, dann **Save**.
4. **Ändern**: Im Fork die Datei **`docs/controls.js`** öffnen, oben rechts auf den **Stift** („Edit this file“) klicken, die Liste `AUX_KANAELE` ändern.

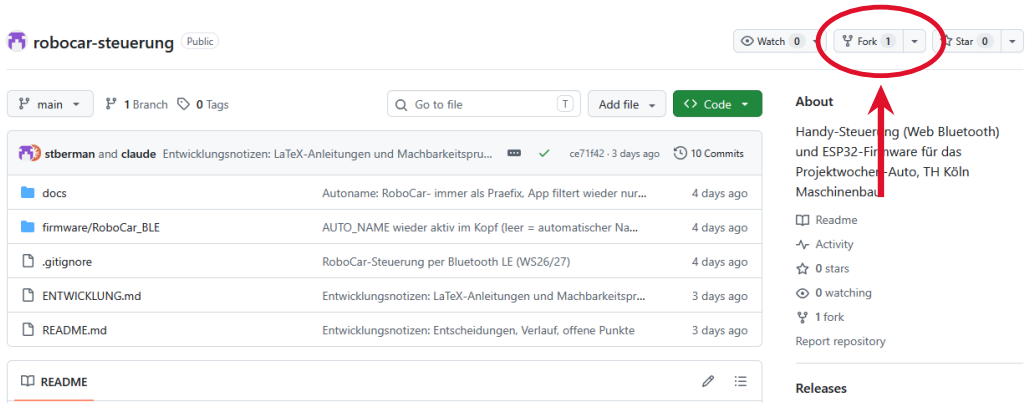


Abbildung 7.2.: Der Knopf „Fork“ auf der Projektseite

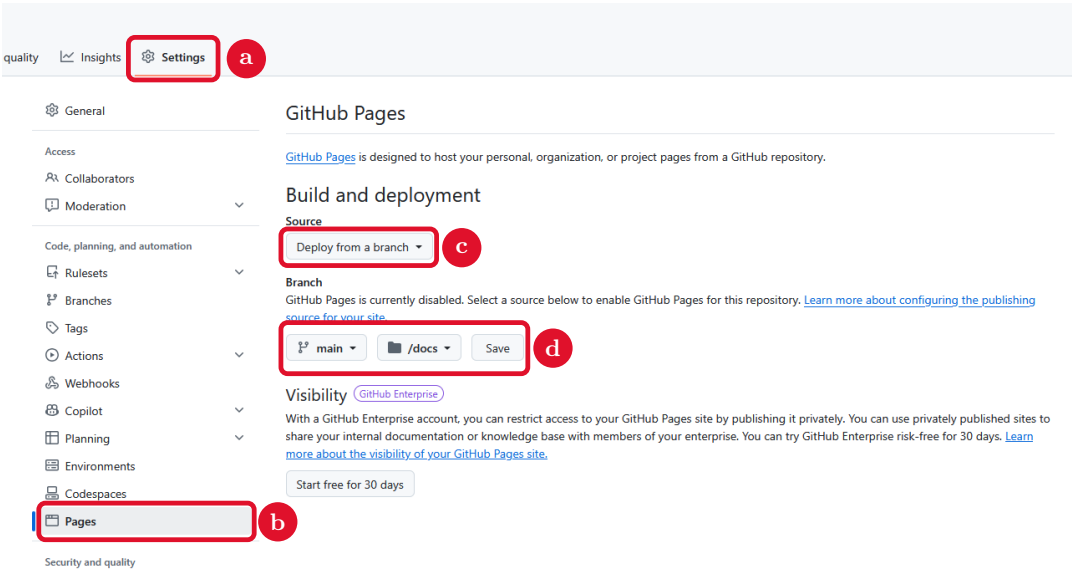


Abbildung 7.3.: In Ihrem Fork: **a** Settings, **b** Pages, **c** Deploy from a branch, **d** main, /docs, Save. Oben links steht hier „beispielAccount“, bei Ihnen Ihr eigener Name.

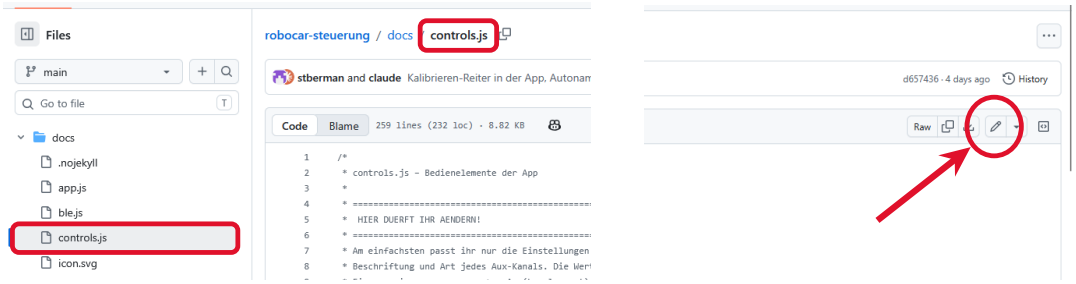


Abbildung 7.4.: Links: Datei **`controls.js`** geöffnet. Rechts: oben rechts über dem Code der Stift zum Bearbeiten.

5 **Speichern**: Oben rechts **Commit changes …** ➙ kurze Beschreibung (z. B. „Kicker-Tasten“) ➙ **Commit changes**.



Abbildung 7.5.: Nach dem Ändern: oben rechts „Commit changes …“ klicken

6 **Warten**: 1–2 Minuten. Den Fortschritt sehen Sie im Reiter **Actions** (grüner Haken = fertig).

7 **Öffnen**: Ihre App liegt jetzt unter `https://IhrName.github.io/robocar-steuerung/`. Diese Adresse auf dem Handy öffnen (Android: Chrome, iPhone: Bluefy) und ggf. zum Startbildschirm hinzufügen.

☞ **Das Handy zeigt noch die alte Version?** Die App speichert sich für den Offline-Betrieb auf dem Handy. Seite neu laden, notfalls die App schließen und neu öffnen. Hilft das nicht: In **`docs/sw.js`** die Zeile `const CACHE = 'robocar-v4';` auf eine neue Nummer ändern (`'robocar-v5'`) und committen. Dann holt sich jedes Handy alles neu.

8. Projekt: Servo-Kicker

Ziel dieses Kapitels: Ein Servo an Ihrem Auto schießt auf Knopfdruck den Ball. Danach bauen Sie das Beispiel Schritt für Schritt aus.

8.1. Was ist ein Servo?

Ein **Servo** ist ein kleiner Motor mit Getriebe und eingebauter Regelung. Er dreht sich nicht endlos, sondern fährt auf einen **Winkel** zwischen 0° und 180° und hält diese Stellung. Den Winkel bekommt er über ein Signal mitgeteilt: Alle 20 ms kommt ein kurzer Puls. Ist der Puls 1 ms lang, fährt der Servo auf 0°, bei 2 ms auf 180°. Dieses Signal erzeugt der ESP32 mit PWM (Pulsweitenmodulation).
Der **SG90** hat drei Kabel: **braun** = Minus (GND), **rot** = Plus (5 V), **orange** = Signal.

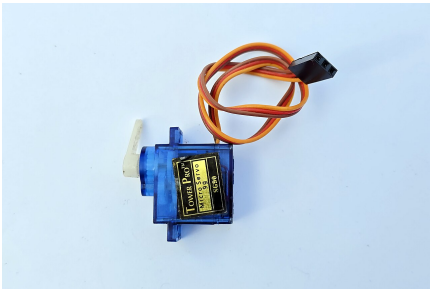


Foto: Suyash Dwivedi, Wikimedia Commons, CC BY-SA 4.0

8.2. Anschließen

1. Auto ausschalten, USB-Kabel abziehen.
2. Mit drei Jumper-Kabeln (Buchse–Buchse) verbinden: Servo **braun** ➞ **GND**, Servo **rot** ➞ **VIN**, Servo **orange** ➞ **D4**.
3. Den Servo stabil am Auto befestigen (Kabelbinder, Heißkleber, 3D-Druck-Halter …) und einen Hebel als Kicker auf die Achse stecken.

▲ Strom:

- Den Servo **nie** an **3V3** anschließen und nie an einen GPIO-Pin als Stromquelle. Ein GPIO liefert nur ca. 20 mA, der Servo braucht beim Anlaufen

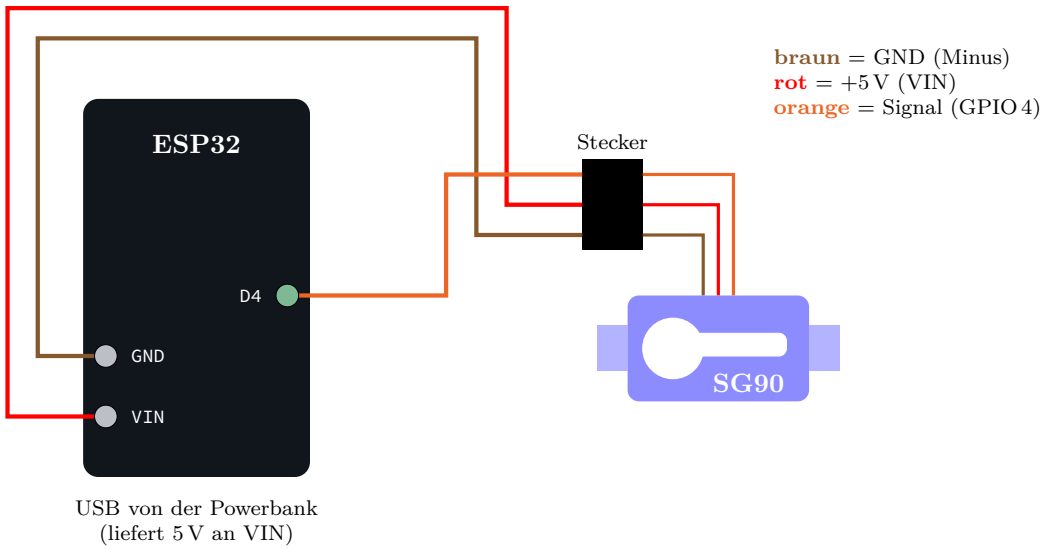


Abbildung 8.1.: Servo am ESP32: Signal an D4 (GPIO 4), Plus an VIN, Minus an GND

bis zu 700 mA.

- **VIN** führt die 5 V aus dem USB-Kabel (Powerbank) direkt durch. Startet der ESP32 bei jedem Schuss neu, reicht der Strom nicht. Dann den Servo direkt aus der Powerbank versorgen (zweiter USB-Ausgang mit aufgeschnittenem Kabel, Betreuer fragen) und **GND von Servo und ESP32 verbinden**.
- Den Servo nie an den 11,1-V-Akku des Autos anschließen. Er geht sofort kaputt.

8.3. Stufe 1: das fertige Beispiel einschalten

In **`aux_user.cpp`** ist ein Kicker-Beispiel schon fertig eingebaut, aber ausgeschaltet.

✎ Aufgabe 3: Kicker einschalten

1. Reiter **`aux_user.cpp`** öffnen. Weiter oben stehen diese zwei Zeilen:

```
#define BEISPIEL_SERVO_KICKER 0   // Servo an GPIO 4, AUX 1 =
    ↪ Schuss
#define BEISPIEL_LICHT        0   // LED an GPIO 16, AUX 2 =
```

```
    ↪ an/aus
```

2. Beim Kicker die `0` durch eine `1` ersetzen:

```
#define BEISPIEL_SERVO_KICKER 1   // Servo an GPIO 4, AUX 1 =
    ↪ Schuss
```

3. Hochladen, Auto mit der App verbinden, **KICK** drücken: Der Servo fährt aus, beim Loslassen wieder ein.

Was passiert im Code? Ein Blick in die Funktion `onAux()`:

```
#if BEISPIEL_SERVO_KICKER
  if (kanal == 1) {
    // wert > 127 heißt: Taste gedrückt (mehr als die Hälfte von 255)
    servoGrad(wert > 127 ? SERVO_SCHUSS_GRAD : SERVO_RUHE_GRAD);
  }
#endif
```

- `#if ... #endif`: Der Code dazwischen wird nur übersetzt, wenn oben die `1` steht. So lässt sich ein Beispiel einfach an- und ausschalten.
- `if (kanal == 1)`: nur reagieren, wenn sich AUX 1 geändert hat.
- `bedingung ? A : B` ist die Kurzform von „wenn Bedingung, dann A, sonst B“.
- `servoGrad(winkel)` ist eine kleine Hilfsfunktion weiter oben in der Datei, die den Winkel in das passende PWM-Signal umrechnet.

In `onFailsafe()` steht außerdem `servoGrad(SERVO_RUHE_GRAD);`: Reißt die Verbindung ab, fährt der Kicker sicher ein.

8.4. Stufe 2: Winkel einstellen

✎ Aufgabe 4: Den Kicker an Ihr Auto anpassen

Ändern Sie in **aux_user.cpp** die beiden Winkel so, dass der Hebel in Ruhe nicht im Weg ist und beim Schuss kräftig ausschlägt:

```
const int SERVO_RUHE_GRAD = 20;    // Kicker eingefahren
const int SERVO_SCHUSS_GRAD = 120; // Kicker ausgefahren
```

Hochladen, testen, wieder ändern. Tipp: Den Hebel erst nach dem ersten Test fest auf die Achse stecken, dann passt die Ruhestellung.

8.5. Stufe 3: Ein Druck = ein Schuss (mit `millis()`)

Beim Beispiel bleibt der Kicker so lange draußen, wie Sie die Taste halten. Schöner wäre: **kurz tippen ➙** Kicker fährt aus und nach 250 ms von selbst wieder ein. Dafür brauchen Sie eine Zeitmessung mit `millis()` (Abschnitt 6).

Die Idee in drei Teilen:

1. In `onAux()`: Beim **Drücken** den Servo ausfahren und die Uhrzeit merken.
2. In `userLoop()`: Regelmäßig nachsehen, ob 250 ms vorbei sind, und dann einfahren.
3. In `onFailsafe()`: Bei Verbindungsabbruch sofort einfahren.

✎ Aufgabe 5: Kicker mit Automatik

Ersetzen Sie den **gesamten** Inhalt von **`aux_user.cpp`** durch den folgenden Code (die Datei liegt auch in der ZIP-Datei im Ordner **`beispiele/`** als **`kicker_automatik.cpp`**, siehe Kapitel 2. Sie können den Inhalt von dort kopieren, statt ihn abzutippen). Der Schalter `BEISPIEL_SERVO_KICKER` wird dabei nicht mehr gebraucht.

```
/*
 * aux_user.cpp - Variante "Kicker mit Automatik"
 *
 * Ein Tipp auf KICK (AUX 1) löst genau EINEN Schuss aus:
 * Der Servo fährt aus und nach SCHUSS_DAUER_MS automatisch wieder ein,
 * auch wenn die Taste länger gedrückt bleibt.
 *
 * Verdrahtung: Servo-Signal (orange) an GPIO 4,
 *              Servo-Plus (rot) an VIN, Servo-Minus (braun) an GND.
 */
#include "aux_user.h"

const int SERVO_PIN = 4;
const int SERVO_RUHE_GRAD = 20;           // Kicker eingefahren
const int SERVO_SCHUSS_GRAD = 120;        // Kicker ausgefahren
```

```
const unsigned long SCHUSS_DAUER_MS = 250; // so lange bleibt der Kicker
    ↪ draußen

// Diese Variablen merkt sich das Programm zwischen den Aufrufen
bool schussLaeuft = false;        // true = Kicker ist gerade draußen
unsigned long schussStart = 0;    // Zeitpunkt (millis) des Schusses
uint8_t letzterWert = 0;          // letzter Wert von AUX 1

// Servo auf einen Winkel (0..180 Grad) stellen
void servoGrad(int grad) {
  grad = constrain(grad, 0, 180);
  int pulsUs = 1000 + grad * 1000 / 180;     // Pulslänge 1000..2000
    ↪ Mikrosekunden
  int duty = (long)pulsUs * 16383 / 20000;   // in PWM-Wert umrechnen (14
    ↪ Bit, 20 ms Takt)
  ledcWrite(SERVO_PIN, duty);
}

void userSetup() {
  ledcAttach(SERVO_PIN, 50, 14);   // PWM mit 50 Hz (= alle 20 ms), 14
    ↪ Bit Auflösung
  servoGrad(SERVO_RUHE_GRAD);
}

void onAux(uint8_t kanal, uint8_t wert) {
  Serial.printf("AUX %u = %u\n", kanal, wert);

  if (kanal == 1) {
    // Nur beim DRÜCKEN schießen (Wert war klein, ist jetzt groß),
    // und nur, wenn nicht schon ein Schuss läuft.
    bool gedrueckt = (wert > 127) && (letzterWert <= 127);
    if (gedrueckt && !schussLaeuft) {
      servoGrad(SERVO_SCHUSS_GRAD);
      schussLaeuft = true;
      schussStart = millis();          // Uhrzeit merken
    }
    letzterWert = wert;
  }
}

void userLoop() {
  // Ist die Schusszeit abgelaufen? Dann Kicker wieder einfahren.
  if (schussLaeuft && millis() - schussStart >= SCHUSS_DAUER_MS) {
    servoGrad(SERVO_RUHE_GRAD);
    schussLaeuft = false;
  }
```

```
}

void onFailsafe() {
  servoGrad(SERVO_RUHE_GRAD);   // Verbindung weg: Kicker einfahren
  schussLaeuft = false;
  letzterWert = 0;
}
```

● **Warum `letzterWert`?** `onAux()` wird bei *jeder Änderung* aufgerufen, also beim Drücken (0 → 255) *und* beim Loslassen (255 → 0). Geschossen werden soll nur beim Drücken. Deshalb merkt sich das Programm den letzten Wert und prüft „vorher losgelassen *und* jetzt gedrückt“. Das nennt man **Flankenerkennung**.

8.6. Stufe 4: Servo stufenlos per Schieber

Statt eines Kickers können Sie mit dem Servo auch eine Klappe oder einen Greifarm bewegen. Mit einem **Schieber** in der App (AUX 3, „ARM“ bzw. „WINKEL“ aus Aufgabe 2) stellen Sie den Winkel direkt ein.

Die Umrechnung erledigt `map(wert, 0, 255, 0, 180)`: Sie rechnet einen Wert aus dem Bereich 0–255 in den Bereich 0–180 um, so wie man Celsius in Fahrenheit umrechnet.

✎ Aufgabe 6: Greifarm

Ersetzen Sie den Inhalt von **`aux_user.cpp`** durch den folgenden Code (in der ZIP-Datei: **`beispiele/servo_schieber.cpp`**):

```
/*
 * aux_user.cpp - Variante "Servo per Schieber"
 *
 * Der Schieber ARM (AUX 3) stellt den Servo-Winkel stufenlos ein,
 * z. B. für einen Greifarm oder eine Klappe.
 *
 * Verdrahtung: Servo-Signal (orange) an GPIO 4,
 *              Servo-Plus (rot) an VIN, Servo-Minus (braun) an GND.
 */
#include "aux_user.h"

const int SERVO_PIN = 4;
const int WINKEL_MIN = 0;     // Winkel bei Schieber ganz links
const int WINKEL_MAX = 180;   // Winkel bei Schieber ganz rechts
```

```
const int WINKEL_START = 90;  // Winkel nach dem Einschalten und bei
    ↪ Failsafe

// Servo auf einen Winkel (0..180 Grad) stellen
void servoGrad(int grad) {
  grad = constrain(grad, 0, 180);
  int pulsUs = 1000 + grad * 1000 / 180;     // Pulslänge 1000..2000
    ↪ Mikrosekunden
  int duty = (long)pulsUs * 16383 / 20000;   // in PWM-Wert umrechnen (14
    ↪ Bit, 20 ms Takt)
  ledcWrite(SERVO_PIN, duty);
}

void userSetup() {
  ledcAttach(SERVO_PIN, 50, 14);
  servoGrad(WINKEL_START);
}

void onAux(uint8_t kanal, uint8_t wert) {
  Serial.printf("AUX %u = %u\n", kanal, wert);

  if (kanal == 3) {
    // map() rechnet einen Bereich in einen anderen um:
    // Schieberwert 0..255  ->  Winkel WINKEL_MIN..WINKEL_MAX
    int grad = map(wert, 0, 255, WINKEL_MIN, WINKEL_MAX);
    servoGrad(grad);
  }
}

void userLoop() {
}

void onFailsafe() {
  servoGrad(WINKEL_START);
}
```

☞ Zum Weiterdenken

- Der Servo soll nur zwischen 30° und 150° fahren? `WINKEL_MIN` und `WINKEL_MAX` ändern.
- Ein zweiter Servo an GPIO 18 für einen weiteren Mechanismus? Zweiten Pin in `userSetup()` mit `ledcAttach(18, 50, 14);` einrichten und `servoGrad()` um einen Parameter für den Pin erweitern. Weitere PWM-Ausgänge sollten ebenfalls **50 Hz** verwenden.

- Kicker *und* Schieber gleichzeitig? Dann brauchen Sie zwei Servos, sonst kämpfen beide Funktionen um denselben.

8.7. Mechanik: So kann ein Kicker aussehen

Wie Sie den Kicker bauen, ist Ihre Sache. Ein einfacher Aufbau ist in Abbildung 8.2 skizziert: Der Servo sitzt vorne in der Mitte, auf seiner Achse steckt ein langer Hebel. In Ruhe liegt der Hebel quer an der Front, beim Schuss dreht er nach vorn und trifft den Ball.

- Den Hebel können Sie aus einem Holzspatel, einem Streifen Kunststoff oder einem 3D-gedruckten Teil bauen und mit der Schraube am Servohorn befestigen.
- Den Servo mit Heißkleber, Kabelbinder oder einem gedruckten Halter am Chassis festmachen. Er muss beim Schuss fest sitzen.
- Die beiden Winkel stellen Sie in Stufe 2 ein. Je länger der Hebel, desto schneller ist seine Spitze.

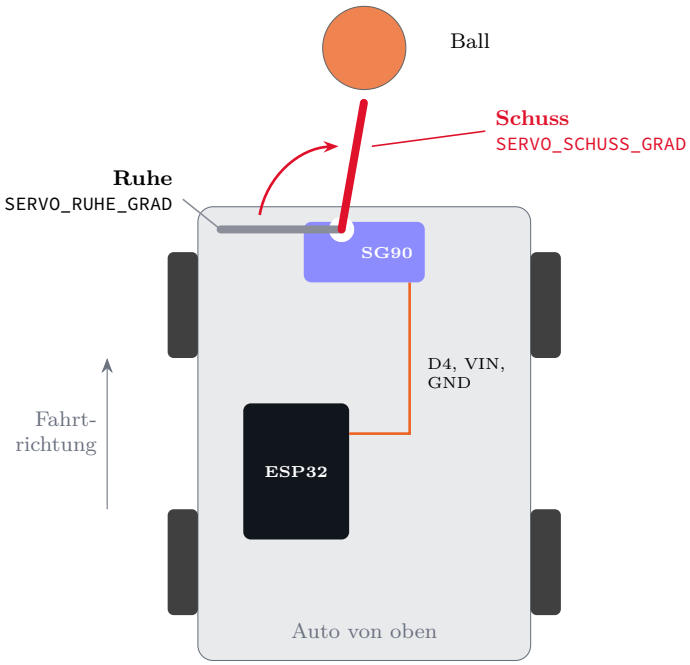


Abbildung 8.2.: Prinzipskizze eines Servo-Kickers von oben. Grau: Hebel in Ruhestellung, rot: Hebel beim Schuss. Die Winkel kommen aus `SERVO_RUHE_GRAD` und `SERVO_SCHUSS_GRAD`.

9. Eigene Ideen: freie Pins und Grenzen

Ziel dieses Kapitels: Sie wissen, welche Pins Sie benutzen dürfen und was ein Pin aushält.

Pins	Verwendung
4, 16, 17, 18, 19, 21, 22, 23	**frei** für Ihre Erweiterungen (Ein- und Ausgang, PWM)
34, 35, 36 (VP), 39 (VN)	nur **Eingang** (z. B. Sensoren), keine eingebauten Pull-up-Widerstände
14, 25, 26, 27, 32, 33	belegt durch die Motoren
2	blaue Status-LED
0, 1, 3, 5, 6–11, 12, 15	**nicht benutzen**: werden zum Starten, Flashen oder für USB gebraucht

▲ Grenzen eines ESP32-Pins:

- Ein Ausgang liefert **3,3 V** und höchstens **ca. 20 mA**. Das reicht für eine LED mit Vorwiderstand (ca. 220 Ω), aber nicht für Motoren, Servos, Relais oder Magnete. Die brauchen eine eigene Versorgung oder einen Treiber (Transistor, MOSFET).
- Ein Eingang verträgt **höchstens 3,3 V**. 5 V-Sensoren (z. B. Ultraschall HC-SR04) brauchen einen Spannungsteiler.
- GND aller Teile immer miteinander verbinden.
- Niemals einen Pin direkt mit GND oder 3V3 verbinden, während er als Ausgang arbeitet (Kurzschluss).

● **Weitere Ideen** Hupe mit einem aktiven Summer (über einen Transistor), Licht (LED mit Vorwiderstand an GPIO 16, Beispiel `BEISPIEL_LICHT` in **`aux_user.cpp`**), ein zweiter Servo, ein Taster als „Tor-Zähler“ am Eingang … Sprechen Sie Bauteile, die nicht im Kit sind, vorher mit den Betreuern ab.

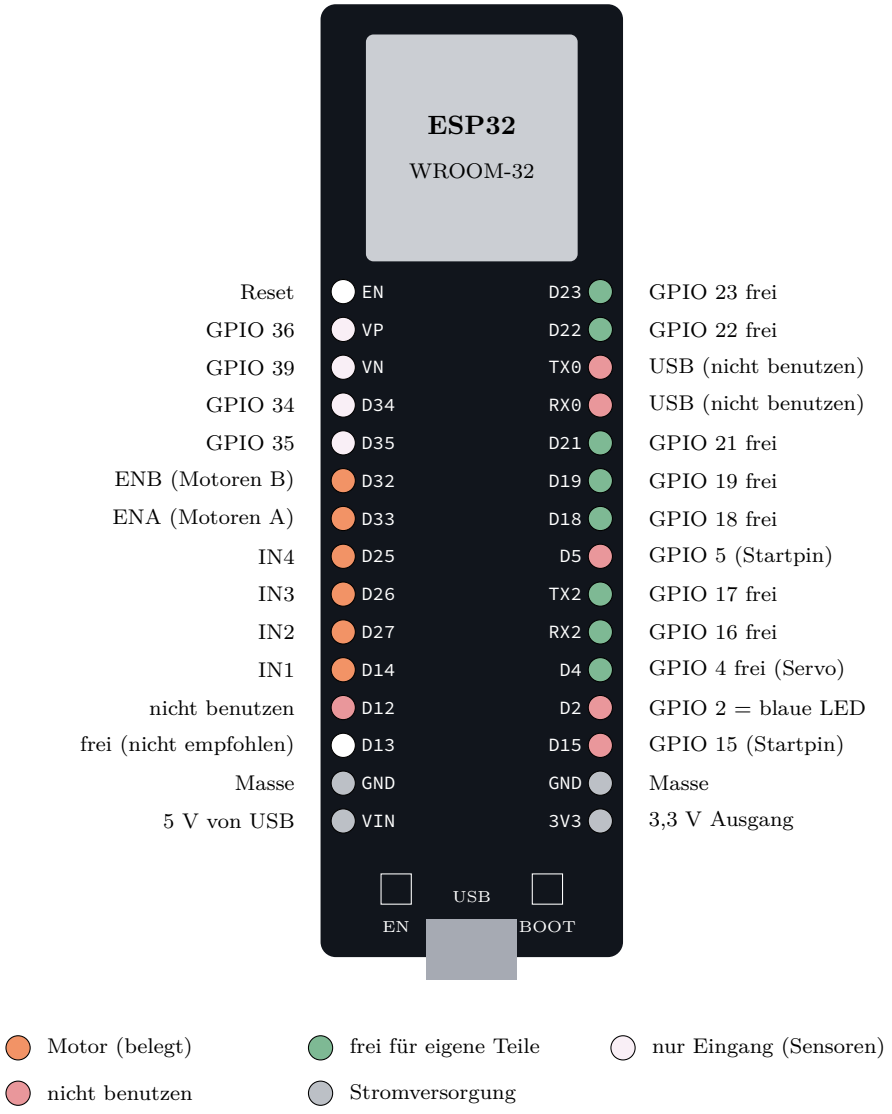


Abbildung 9.1.: ESP32 DevKit V1 von oben (USB-Buchse unten). Innen steht die Beschriftung auf dem Board, außen die Bedeutung. Achtung: GPIO 16 und 17 sind auf dem Board mit **RX2** und **TX2** beschriftet.

10. Fehlersuche

Problem	Ursache	Lösung
Arduino IDE / Flashen		
`Falsches Board!` beim Kompilieren	falsches Board gewählt	Werkzeuge › Board › esp32 › ESP32 Dev Module
`Preferences.h: No such file`	falsches Board oder alter Zwischenspeicher	Board prüfen; sonst IDE schließen, neu starten, nochmal
`NimBLEDevice.h: No such file`	Bibliothek fehlt	NimBLE-Arduino 2.x installieren (Kap. 4)
Kein Port im Menü	Ladekabel ohne Datenleitung oder Treiber fehlt	anderes Kabel; Treiber CP2102/CH340 installieren
`Failed to connect`, `Wrong boot mode`	Board geht nicht in Flash-Modus	beim `Connecting...` BOOT gedrückt halten
`Access is denied`, `Port busy`	Port von anderem Programm belegt	Seriellen Monitor/zweite IDE schließen
Serieller Monitor zeigt Zeichensalat	falsche Baudrate	unten rechts **115200** wählen
App / Bluetooth		
„Dieser Browser kann kein Bluetooth“	falscher Browser	Android: Chrome; iPhone: Bluefy
Auto erscheint nicht in der Liste	Auto aus, schon verbunden, oder Name ohne `RoboCar-`	LED muss langsam blinken; anderes Handy trennen; Auto aus/an
App bleibt nach Änderung leer	Tippfehler in **`controls.js`**	Hochkommas und Kommas prüfen; am Laptop mit F12 Konsole öffnen, dort steht der Fehler
Handy zeigt alte App-Version	Zwischenspeicher	neu laden; `CACHE` in **`sw.js`** hochzählen
Auto hält beim Fahren kurz an	Verbindung bricht ab (Failsafe)	näher heran; Handy nicht in Hosentasche; Energiesparmodus aus

Problem	Ursache	Lösung
Auto / Servo		
Motor dreht falsch herum	Verkabelung	⚙ › Allgemein: Häkchen setzen
Auto zieht zur Seite	Motoren unterschiedlich	⚙ › Kalibrieren › Trim
Motoren brummen, drehen aber nicht	Anlaufschwelle zu klein	⚙ › Kalibrieren › Anlaufschwelle erhöhen
ESP32 startet bei Servobewegung neu	Strom reicht nicht	Servo extra versorgen, GND verbinden
Servo zittert	schlechte Masseverbindung oder zu wenig Strom	Kabel prüfen, GND sicher verbinden
Servo tut nichts	falscher Pin, Beispiel nicht eingeschaltet, falscher AUX-Kanal	Serieller Monitor: kommt `AUX 1 = 255` an?

☞ **Vorgehen bei der Fehlersuche**

1. Nur **eine** Sache auf einmal ändern, dann testen.
2. Mit `Serial.println()` ausgeben lassen, ob eine Stelle im Code überhaupt erreicht wird.
3. Wenn gar nichts mehr geht: Originalprogramm aufspielen (Abschnitt 5.7) und von vorn beginnen.

A. Spickzettel

Einmalig installieren

- Arduino IDE 2.x
- Boardverwalter: **esp32 by Espressif** 3.x
- Bibliotheken: **NimBLE-Arduino** (h2zero) 2.x
- Programm: https://github.com/stberman/robocar-steuerung → Code → Download ZIP → nach `C:\RoboCar` entpacken (Kap. 2)

Werkzeuge-Menü

- Board: **ESP32 Dev Module**
- Port: COMx (neu beim Einstecken)
- Upload Speed: 921600 (sonst 115200)
- Erase All Flash: **Disabled**

Flashen

- Akku aus, USB an Laptop
- → Hochladen; bei `Connecting...` ggf. BOOT halten
- fertig bei `Hard resetting`
- Serieller Monitor: 115200 baud

Wo ändere ich was?

- Name: **`RoboCar_BLE.ino`** → `AUTO_NAME`
- Eigene Funktionen: **`aux_user.cpp`**
- Tasten der App: **`docs/controls.js`** → `AUX_KANAELE`
- Nicht ändern: **`motor.cpp`**, **`ble_link.cpp`**, **`ble.js`**

Ihre Funktionen in `aux_user.cpp`

- `userSetup()` – einmal beim Start
- `onAux(kanal, wert)` – AUX geändert (1–4, 0–255)
- `userLoop()` – alle 10 ms
- `onFailsafe()` – Verbindung weg
- **kein** `delay()` – `millis()` benutzen

Pins

- frei: **4, 16 (RX2), 17 (TX2), 18, 19, 21, 22, 23**
- nur Eingang: 34, 35, 36, 39
- Pin: 3,3 V, max. 20 mA
- Servo: braun GND, rot VIN, orange D4

App testen

- Laptop: `index.html?demo`
- Laptop + Auto: `python -m http.server 8000` → `localhost:8000`
- Handy: Fork → Settings → Pages (main, /docs)
- Alte Version? `CACHE` in **`sw.js`** erhöhen

App-Einstellungen (⚙)

- Allgemein: Gruppe, Tempo, Motoren umdrehen → **Speichern**
- Kalibrieren: wirkt sofort; Anlauf → Trim → Kurve

B. Verkabelung des Autos

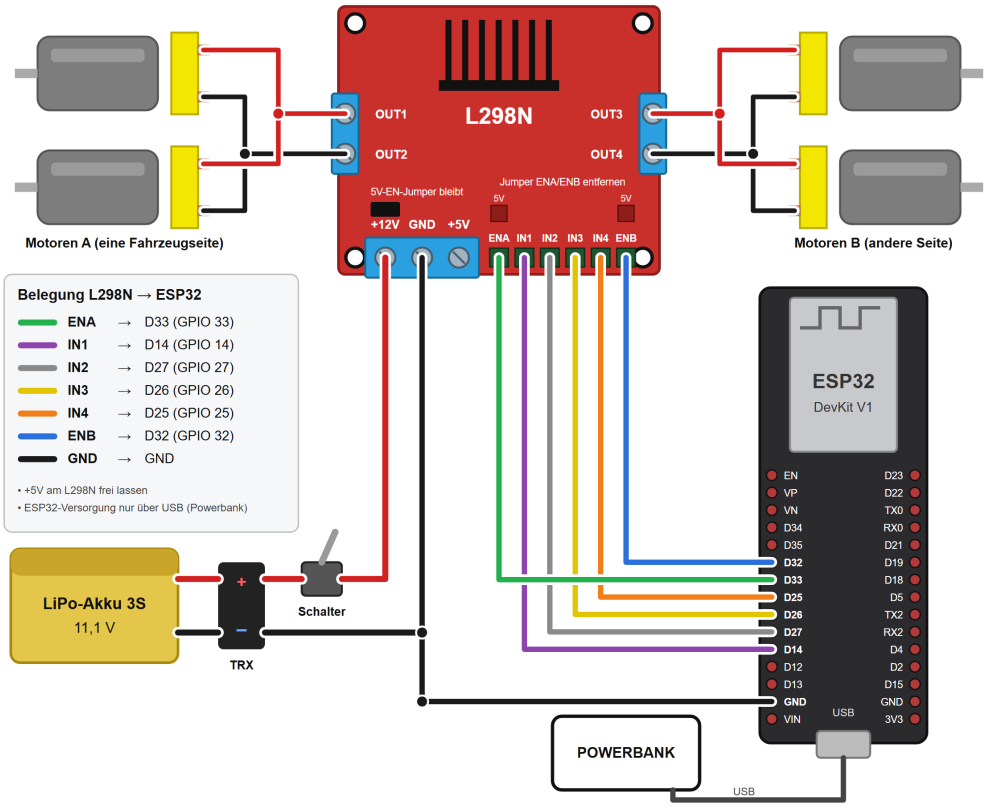


Abbildung B.1.: Verkabelungsplan WS26/27 (aus der Aufbauanleitung)

C. Bildnachweis

- SG90-Servo: Suyash Dwivedi, „Tower Pro SG90 micro servo motor“, Wikimedia Commons, Lizenz CC BY-SA 4.0, https://commons.wikimedia.org/wiki/File:Tower_Pro_SG90_micro_servo_motor.jpg
- Alle weiteren Grafiken und Screenshots: eigene Darstellung, TH Köln.